

生成 AI 利用申告書

- 氏名：塩澤 匠生
- 学籍番号：25G1065
- プログラムファイル名：HCK02_02.ino / HCK02_02.pde
- 使用した AI ツール：ChatGPT (GPT-5)、Claude Code (Opus 4.7)

1. 何に使ったか

提出課題 2 は「前回課題（ハッカソン 1 第 1 回）で作成した楽曲を Processing で再生しつつ、マイクから入った音の波形を Processing のウィンドウに描画する」課題である。授業の例題 1 では Arduino から Processing へ 1 バイトの値を送り、円の色を切り替える書き方 (`Serial.write` と `serialEvent + read()`) までを学んだが、波形のように「サンプルを連続して送って時系列に並べて表示する」書き方は扱われていなかった。そこで

- Arduino から Processing へマイクのサンプルを連続して渡す方法
- Processing 側で受け取ったサンプルを時系列の波形として描画する方法（バッファの持ち方、描画ループとの分離）

の 2 点を生成 AI に相談した。楽曲再生（Minim を使う部分）は前回課題で書いたコードを流用したため、AI に相談していない。

2. AI への指示（プロンプト）

Arduino でマイクの値を `analogRead` して、Processing のウィンドウに時系列の波形として描きたいです。

- 授業の例題1 では `Serial.write` で 1 バイト送って、Processing 側で `serialEvent` から `read()` を呼ぶ書き方を習いました
- 今回は値を連続で送り続け、ウィンドウの左から右に波形が並ぶように描きたいです

Arduino 側はどう送るのが扱いやすいですか？ また Processing 側では受信したサンプルをどんなバッファに貯めて、どう毎フレーム描けばよいですか？

3. AI の出力の使い方

- ☐ そのまま使った
- ☒ 一部修正して使った
- ☐ 参考にしただけ

→ 上で選んだ理由：

AI からは、

- Arduino 側は `Serial.println(d)` で「1 行 1 サンプル」のテキストとして送ると、Processing 側で改行区切りで切り出せて扱いやすい
- Processing 側は `port.bufferUntil('\n')` と `serialEvent` を組み合わせ、改行が来たときだけ

`readStringUntil` で 1 行受け取って整数に変換する

- 描画は循環バッファ (`int[]` と書き込みインデックス) に値を貯め、`draw()` の中で「最新位置から左方向に直近 N サンプル」を読み出して線で結ぶ

という方針が出てきた。例題 1 の `serialEvent` の使い方とも整合していたのでこの骨格はそのまま採用した。一方で、最初に提示されたサンプルは「2 バイトのバイナリで送る」「描画は `draw()` の中で `port.read()` を呼ぶ」書き方で、デバッグしづらく、また値の取りこぼしが起きそうだったので採用しなかった。

4. 正しいと確認した方法

- Arduino IDE で `HCK02_02.ino` がコンパイルでき、シリアルモニタ (921600 bps) に整数が 1 行ずつ流れることを確認した
- Processing IDE で `HCK02_02.pde` を起動し、無音時はほぼ平らな線、マイクに声を出すと白い線が縦に振れる様子を観察した
- キー p を押すと前回課題 (ハッカソン 1 第 1 回) と同じ楽曲が PC のスピーカから流れ、その音をマイクが拾って波形がより大きく振れることを確認した
- キー 1~6 で音色を切り替えてから p を押すと異なる音色で再生され、波形の見え方も変わることを確認した
- Processing のコードを読み直し、循環バッファのインデックス計算 (`writeIdx - DISPLAY_SAMPLES + samples.length) % samples.length` がなぜ「右端が最新サンプル」になるかを自分で説明できるようにした

5. 問題や間違い

- ☒ あった
- ☐ なかった

最初に AI が出してきた Processing コードは、`port.read()` を `draw()` の中で呼ぶ書き方だった。これだと値の取りこぼしや描画ループとのタイミングずれが起きやすそうだったので、授業の例題 1 と同じ `serialEvent` 方式に揃えた。また、起動直後にゴミ文字が混ざってきたとき `Integer.parseInt` が例外を投げる場合があったため、`NumberFormatException` を握りつぶす `try-catch` を入れて落ちないようにした。

6. 自分で工夫した点

- Arduino 側 (`HCK02_02.ino`) は、提出課題 1 のプロッタ用ガイド (0, 1023) を取り除いて `Serial.println(d)` 1 本に絞りと、Processing が一番シンプルに受け取れる形にした
- Processing 側 (`HCK02_02.pde`) は、前回課題の楽曲再生コード (`HackInstrument` クラス、メロディ・拍・音量の配列、`Minim` 関連のセットアップ) をそのまま組み込み、波形描画と楽曲再生が同じウィンドウ・同じキー操作 (p, 1-6) で扱えるようにした
- 循環バッファの長さをウィンドウ幅 `width` と揃え、画面幅とサンプル数の対応をシンプルにした
- 直近の `DISPLAY_SAMPLES` 個 (60 サンプル) だけを画面いっぱいに引き伸ばして描画することで、約 6 周期程度のゆったりした波形になり、見やすさを確保した

参考文献

- 有本泰子, 佐波孝彦 「ハッカソン 1 第 2 回」 講義資料 (HCK1_02.pdf, HCK1_02_ex.pdf)
- 生成 AI 利用申告書テンプレート (ハッカソン 1 講義資料)
- Arduino Reference: `Serial.println`, `analogRead`
<https://docs.arduino.cc/language-reference/>
- Processing Reference: `Serial`, `bufferUntil`, `serialEvent`
<https://processing.org/reference/libraries/serial/>
- Minim Reference: `AudioOutput`, `Oscil`, `Line`
<https://code.compartmental.net/minim/>